ProcurePulse: Deliverable 1

Problem Definition & Data Foundations

Hassan Ashraf 24L3073
Sharjeel Shahid 24L3055
Zunaira Tahir 24L3038

October 2026

Contents

1 Introduction **1**

2 Problem Definition **2**
2.1 Problem Statement . . . 2
2.2 Target Audience . . . 3
2.3 AI Justification . . . 3

3 Data Collection **4**
3.1 Data Sources . . . 4
3.2 Collection Method . . . 5
3.3 Raw Data Description . . . 6
3.4 Data Quality Issues Found . . . 8

4 Data Cleaning **9**
4.1 Overview . . . 9
4.2 Cleaning Pipeline Summary . . . 9
4.3 Methodological Justifications . . . 10

5 Feature Strategy and Exploratory Data Analysis **11**
5.1 Feature Engineering Strategy . . . 11
5.2 Calendar Encoding and Categoricals . . . 11
5.3 Chronological Dataset Partitioning . . . 12
5.4 Outlier Capping and Feature Scaling . . . 12
5.5 Exploratory Data Analysis . . . 12

6 Conclusion **18**

1 Introduction

ProcurePulse is an AI-assisted tool that helps businesses decide *when* to buy metals and whether a supplier's price is fair. It forecasts the prices of gold and other metals traded on COMEX, attaches a prediction interval to each forecast, and compares a supplier's quote with that forecast to recommend **Buy**, **Hold** or **Negotiate**. The project also tests whether external economic signals, such as the US dollar, interest rates, oil and inflation, improve forecast accuracy.

This report covers the first deliverable, *Problem Definition and Data Foundations*. Its objectives are to:

1. define the real-world problem, the target users and why machine learning is needed (Phase 1);
2. collect 16 years of daily market data from public sources with reproducible, logged Python scripts (Phase 2);
3. clean the data, engineer features and explore it visually, producing a dataset ready for modelling (Phase 2).

The report is organised as follows. Section 2 defines the problem, the target audience and the AI justification. Section 3 describes the data sources, how the data was collected and what the raw data looks like. Section 4 explains how the data was cleaned, and Section 5 presents the feature strategy and the exploratory data analysis. Section 6 summarises the results and the plan for the next deliverable. All code is in the `data_scripts/` folder of the project's GitHub repository.

2 Problem Definition

2.1 Problem Statement

Businesses that make products from metals, such as jewellers, electronics assemblers and metal fabricators, buy their raw materials at prices they do not control. These prices are set on commodity exchanges such as COMEX and change every trading day. Our own data (Figure 1) shows how large these changes are: gold futures traded at about \$1,050 per ounce in late 2015 and rose above \$5,000 per ounce in early 2026, while copper roughly tripled from its 2016 low. Movements over a single day can also be severe: the largest one-day falls in our 2010–2026 data are -10.8% for gold, -22.3% for copper and -31.3% for silver.

For a buyer, this volatility creates two practical problems:

- **Timing.** Most small and medium-sized businesses buy when stock runs low or on a fixed schedule, at whatever the price is that day. Buying a few weeks earlier or later can change the cost of the same order by several percent, which can be the whole profit margin.
- **Judging a supplier's quote.** Suppliers quote the market price plus a markup. Without a view of where the price is likely to go, the buyer cannot tell whether a quote is fair, generous or inflated.

Large firms handle this with in-house analysts and hedging desks. Smaller firms usually rely on intuition, news headlines or the supplier's own advice, which is not neutral.

ProcurePulse addresses this gap. It forecasts gold and other metal prices from COMEX data, gives each forecast a *prediction interval* (a range the price is expected to fall within, with a stated probability), and compares a supplier's quote against that forecast to recommend one of three actions: **Buy** (the quote is good relative to the expected price), **Hold** (prices are expected to fall, so waiting is likely cheaper) or **Negotiate** (the quote is high relative to the forecast range).

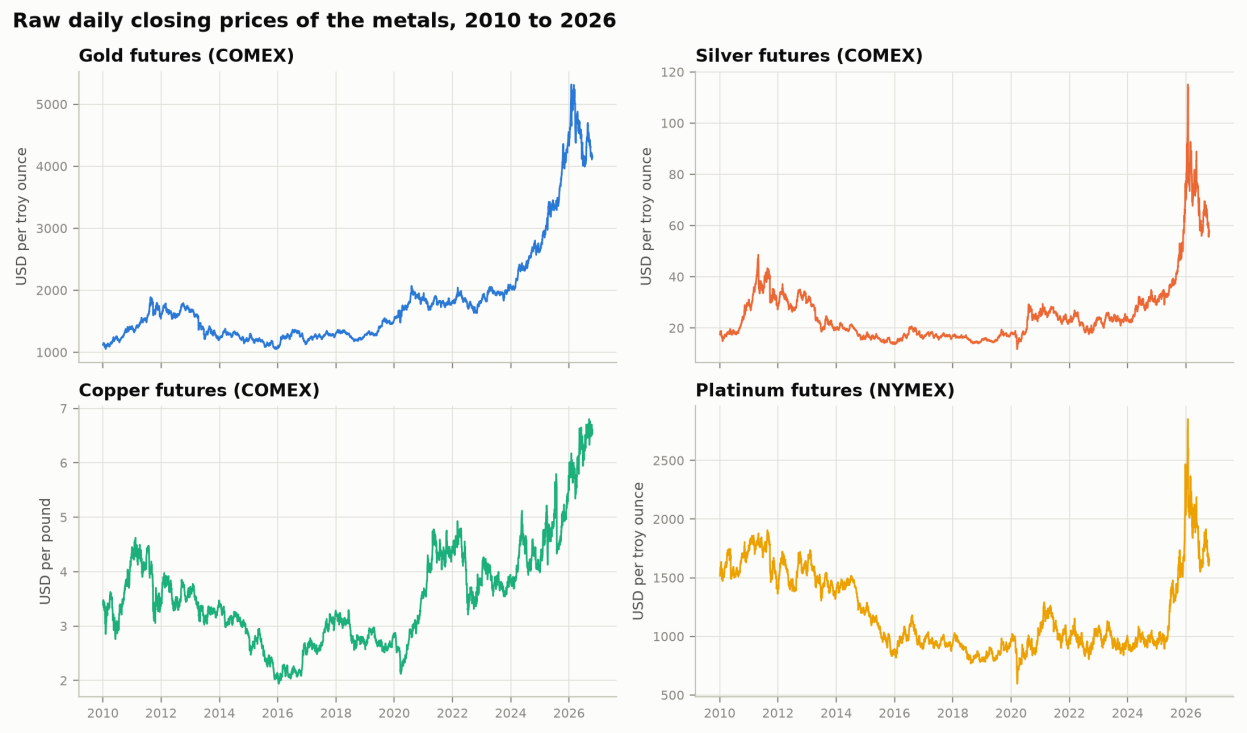


Figure 1: Daily closing prices of COMEX gold, silver and copper futures and NYMEX platinum futures, January 2010 to October 2026 (raw data).

2.2 Target Audience

The primary users are **procurement managers and owners of small and medium-sized enterprises (SMEs) that buy metals regularly** but do not have their own market analysts:

- jewellery manufacturers and gold traders (gold, silver, platinum);
- electronics and electrical-goods assemblers (copper, gold, silver);
- metal fabricators and workshops (copper).

Example user (hypothetical). Sara manages purchasing for a 40-person jewellery workshop that buys about 2 kg of gold each month. Each week she receives quotes from two suppliers and must decide whether to accept one, wait or push for a lower price. She has no time to follow the markets in detail. ProcurePulse gives her a forecast with a clear range and a one-word recommendation she can act on.

A secondary audience is students and analysts who want a transparent, reproducible comparison of forecasting models for commodity prices, including whether external economic signals improve accuracy.

2.3 AI Justification

A rule-based system would encode fixed instructions such as "buy when the price falls 5% below its 30-day average". Such rules fail for this problem for four reasons:

1. **The patterns are not fixed.** Metal prices move through different regimes: long calm periods, sharp trends and sudden shocks (for example March 2020 and early 2026 in Figure 1). A threshold that works in one regime fails in the next, and someone would have to keep rewriting the rules by hand.

2. **Many signals interact.** Gold tends to move against the US dollar and real interest rates, while oil prices and inflation also play a role. The strength of these relationships changes over time and is non-linear, which is very hard to express as hand-written rules.

3. **Uncertainty must be measured, not guessed.** The Buy/Hold/Negotiate decision depends on how likely the price is to move, not only on a single predicted value. Statistical and machine learning models can learn this uncertainty from historical data and express it as a prediction interval; a fixed rule cannot.

4. **The data is large and keeps growing.** More than 4,000 trading days across eight series are far too many for a person to turn into reliable rules, while a model can be retrained automatically as new data arrives.

We therefore treat price forecasting as a **supervised time-series learning problem**. In later deliverables we will compare models of increasing complexity on the same chronological train/validation/test split:

- a **naive baseline** (tomorrow's price equals today's) and **ARIMA**, a classical statistical model [2];
- **Prophet** [10] and the gradient-boosted tree model **XGBoost** [3];
- a **Long Short-Term Memory (LSTM)** recurrent neural network [5], which can learn longer-range patterns in sequences.

Each model will be evaluated with standard forecasting error measures and the coverage of its prediction intervals [6], with and without the external signals, to test whether those signals actually improve accuracy. The approach is only worth using if the learned models beat the naive baseline, and this comparison is what will show it.

3 Data Collection

3.1 Data Sources

We collected daily data from 1 January 2010 to 20 October 2026 for four metals and four external signals (Table 1). All sources are public and free. Metal prices are *continuous front-month futures* prices, which is the benchmark that physical metal suppliers price against.

Column	Description	Source (ID)	Frequency	Rows
gold	COMEX gold futures, USD/oz	Yahoo Finance (GC=F)	Daily	4,225
silver	COMEX silver futures, USD/oz	Yahoo Finance (SI=F)	Daily	4,224
copper	COMEX copper futures, USD/lb	Yahoo Finance (HG=F)	Daily	4,225
platinum	NYMEX platinum futures, USD/oz	Yahoo Finance (PL=F)	Daily	4,223
dxy	US Dollar Index	Yahoo Finance (DX-Y.NYB)	Daily	4,226
wti	WTI crude oil futures, USD/bbl	Yahoo Finance (CL=F)	Daily	4,225
us10y	10-year US Treasury yield, %	Yahoo Finance (^TNX)	Daily	4,223
cpi	US Consumer Price Index (SA)	BLS (CUSR0000SA0)	Monthly	201

Table 1: Data sources. Rows are the number of observations downloaded between January 2010 and October 2026.

Why these external signals. Each one is a widely cited driver of metal prices, and testing whether they improve forecasts is part of our project:

- **US Dollar Index:** metals are priced in US dollars, so a stronger dollar makes them more expensive for other buyers and tends to push prices down.
- **10-year Treasury yield:** gold pays no interest, so when interest rates rise, holding gold becomes less attractive compared with bonds.
- **WTI crude oil:** oil feeds into mining and transport costs and into general inflation.
- **Consumer Price Index:** gold is often bought as a hedge against inflation, which the CPI measures.

3.2 Collection Method

Collection is done by three Python scripts in `data_scripts/`, run in this order:

1. `fetch_metals.py` downloads the four metal futures from Yahoo Finance using the `yfinance` library [1] and saves one CSV file per metal.
2. `fetch_external.py` downloads the dollar index and oil from Yahoo Finance, and the 10-year yield and CPI from FRED [4].
3. `merge_sources.py` joins all eight series on their date into one table, `merged_raw.csv`.

Every download is retried up to three times, and each step writes a line to `data_scripts/logs/collection.log` with the number of rows, the date range and the number of missing values. The log is our evidence of collection; a copy is included in the repository as `report/evidence/collection_log.txt`. Raw data files are not uploaded to GitHub; anyone can recreate them by running the three scripts. All shared settings (tickers, start date and file paths) are kept in one file, `config.py`.

Backup sources. During collection, FRED repeatedly refused or timed out the connection from our network. Instead of failing, `fetch_external.py` automatically switched to backup sources, and recorded the switch in the log:

- the 10-year yield was taken from Yahoo Finance's `^TNX` (the CBOE 10-year Treasury yield index, quoted in percent), which tracks FRED's `DGS10` series closely;

- CPI was taken directly from the US Bureau of Labor Statistics API [11], series `CUSR0000SA0`. This is the same seasonally adjusted CPI that FRED republishes as `CPIAUCSL`. The free API returns at most ten years per request, so the script requests 2010–2019 and 2020–2026 separately.

The output of the collection scripts is shown below and in Figure 2.

```
gold (GC=F): 4225 rows, 2010-01-04 to 2026-10-20, 0 missing
silver (SI=F): 4224 rows, 2010-01-04 to 2026-10-20, 0 missing
copper (HG=F): 4225 rows, 2010-01-04 to 2026-10-20, 0 missing
platinum (PL=F): 4223 rows, 2010-01-04 to 2026-10-20, 0 missing
dxy (DX-Y.NYB): 4226 rows, 2010-01-04 to 2026-10-20, 0 missing
wti (CL=F): 4225 rows, 2010-01-04 to 2026-10-20, 0 missing
us10y: FRED unavailable, using backup source...
us10y (Yahoo ^TNX): 4223 rows, 2010-01-04 to 2026-10-20, 0 missing
cpi: FRED unavailable, using backup source...
cpi (BLS CUSR0000SA0): 201 rows, 2010-01-01 to 2026-09-01, 1 missing
merged_raw.csv: 4300 rows x 8 columns, 2010-01-01 to 2026-10-20
```

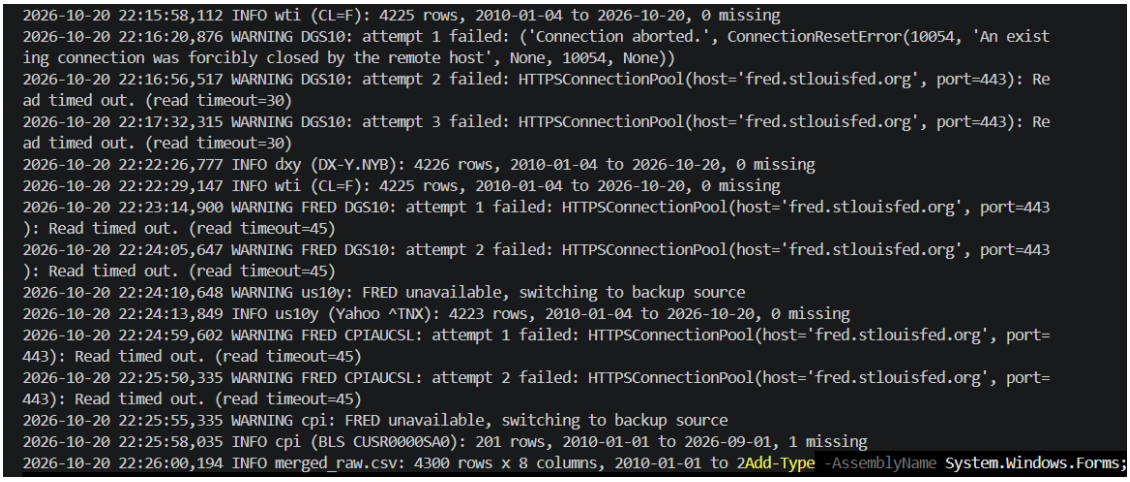


Figure 2: Running the collection scripts in the terminal.

3.3 Raw Data Description

The merged raw table has **4,300 rows and 8 columns**, one row for every date that appears in at least one source, from 1 January 2010 to 20 October 2026. Gaps are deliberately left empty at this stage; they are handled in cleaning (Section 4). Figures 1 and 3 show the raw series, and Table 2 and Figure 4 summarise the missing values.

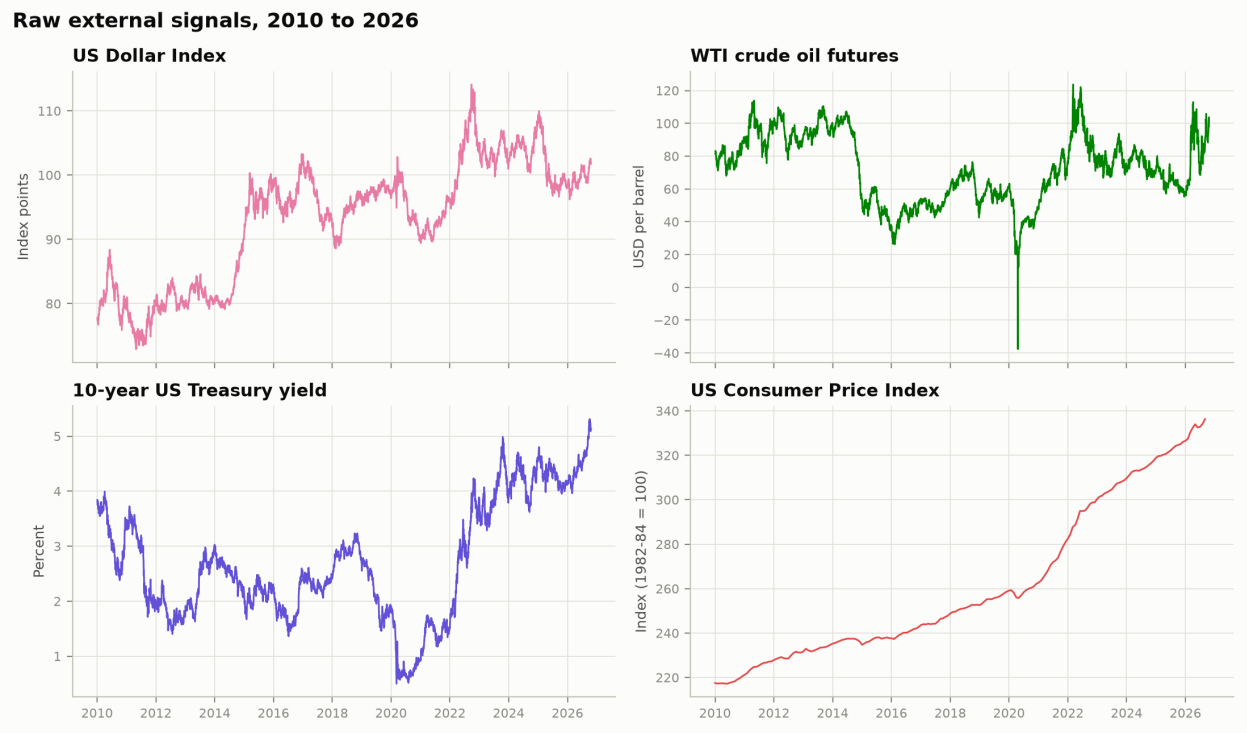


Figure 3: Raw external signals, 2010–2026. Note the WTI oil futures price falling below zero in April 2020.

Column	Missing rows	Share of 4,300 rows
`gold`	75	1.7%
`silver`	76	1.8%
`copper`	75	1.7%
`platinum`	77	1.8%
`dxy`	74	1.7%
`wti`	75	1.7%
`us10y`	77	1.8%
`cpi`	4,100	95.3%

Table 2: Missing values per column in `merged_raw.csv`.

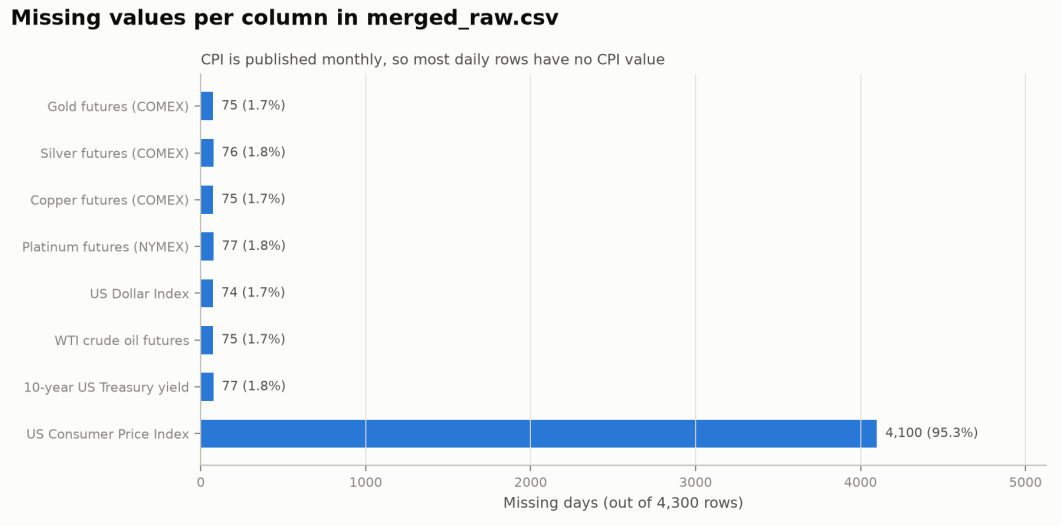


Figure 4: Missing values per column in the raw merged data.

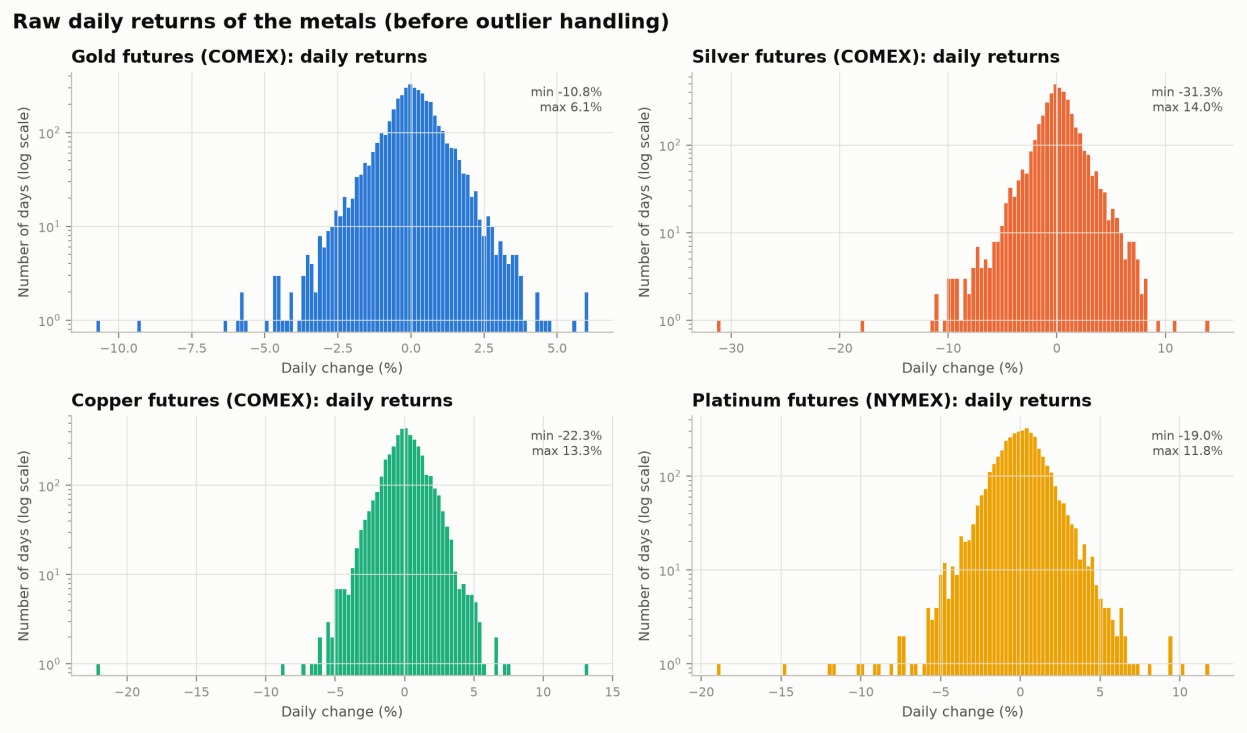


Figure 5: Distribution of raw daily returns of the four metals (log scale). The long tails show rare extreme days.

3.4 Data Quality Issues Found

Inspecting the raw data revealed the following issues, which are handled in Section 4:

- **Gaps from different calendars.** About 75 rows (1.7–1.8%) are missing in each daily series because markets close on different holidays, so a date present in one source can be absent in

another.

- **Mixed frequencies.** CPI is published monthly, so 95% of the daily rows have no CPI value. One month is also missing in the BLS series itself.
- **An impossible price.** WTI oil futures closed at about −$37 per barrel on 20 April 2020 (Figure 3). This was a real event caused by an expiring contract, but a negative price cannot be used for percentage returns.
- **Extreme daily moves.** The largest one-day falls are −10.8% for gold, −19.0% for platinum, −22.3% for copper and −31.3% for silver (Figure 5). Some are genuine market crashes; others may come from the way a continuous futures series jumps from one contract to the next. They must be detected and treated as outliers.

4 Data Cleaning

4.1 Overview

The raw merged dataset (`merged_raw.csv`) contains 4,300 rows and 8 columns, spanning from 1 January 2010 to 20 October 2026. Because each financial and macroeconomic series originates from independent reporting calendars and publishing frequencies, the merged data contains cross-calendar gaps, monthly reporting intervals, an unphysical negative contract price, and severe return outliers. We developed a sequential, leak-free cleaning pipeline in `data_scripts/clean_data.py` to resolve these issues systematically. All operations, row transitions, and column-level missing values were recorded during execution in our audit trail, preserved in `report/evidence/cleaning_log.txt`.

4.2 Cleaning Pipeline Summary

Table 3 outlines the sequence of cleaning operations, the respective methods applied, and the before-and-after observation counts across the dataset.

Issue	Method	Before	After
Duplicate dates	Keep last observation	0	0
Missing CPI (monthly, dated the 1st)	Forward-fill	4,100	0
Non-trading days (all four metals missing)	Drop rows	4,300	4,225 rows
Short gaps in single series	Time interpolation, limit 3 consecutive trading days, inside only	10	0
Remaining missing	Dropna	0 rows removed	0
Non-positive prices	Set NaN then time-interpolate	1 value (WTI)	0
Outlier days (3 × IQR on daily returns)	Flagged via `is_outlier`, not deleted	129 days flagged	4,225

Table 3: Data cleaning pipeline summary showing the resolution method and effect of each sequential step.

The individual outlier counts detected across the price series are 31 for gold, 57 for silver, 12 for copper, 24 for platinum, 6 for the US Dollar Index (`dxy`), and 45 for WTI crude oil (`wti`). Across the entire series, 129 days are flagged as containing an outlier in at least one asset, representing about 3.0% of the 4,225 trading days.

4.3 Methodological Justifications

Order of CPI forward-filling. Consumer Price Index data is published monthly on the 1st calendar day of each month. Because the 1st of a month frequently coincides with a weekend or public holiday when exchange markets are closed, dropping non-trading days prior to forward-filling would discard the row bearing the newly released CPI figure, permanently losing that inflation update. By forward-filling the CPI column first, the newly published index level is propagated forward onto active subsequent trading dates. This forward-fill strictly projects past, officially published figures into the future and introduces zero lookahead leakage.

Time interpolation for short gaps. To bridge brief isolated exchange holidays where a single series is absent while other markets trade, we apply time-weighted interpolation, up to 3 consecutive missing trading days, inside only. Restricting interpolation to strictly inside boundaries ensures that missing values are only interpolated between known observations, preventing unconstrained extrapolation at the historical edges of the dataset.

Return-based outlier detection. We detect statistical outliers on daily percentage returns rather than raw price levels. Gold futures closed at about \$1,118 per ounce on 4 January 2010 and reached about \$4,134 on 20 October 2026. A fixed threshold applied to raw price levels would flag most recent trading days simply due to long-term nominal price growth. Daily returns are centred near zero and much closer to stationary than prices, providing a stable baseline for statistical dispersion metrics over a sixteen-year horizon.

Three-times interquartile range threshold. We adopt a $3 \times$ IQR boundary ($[Q_1 - 3 \times \text{IQR}, Q_3 + 3 \times \text{IQR}]$) rather than the standard Tukey $1.5 \times$ IQR rule. Empirical financial returns possess heavy tails and volatility clustering; setting a $1.5 \times$ IQR limit would label normal, high-volatility market sessions as anomalies. A $3 \times$IQR threshold reserves outlier labels for extreme tail moves.

Flagging rather than row deletion. We record outlier occurrences using a binary indicator column (`is_outlier`) without removing or capping rows during data cleaning. Deleting rows breaks the contiguous chronological spacing required by autoregressive models and loses valid information from genuine historical market dislocations, such as the market turbulence of March 2020. Flagging preserves full historical variance while allowing downstream models to identify shock sessions explicitly.

Impossible non-positive values. Asset prices cannot be zero or negative in return computations, as division by non-positive bases yields undefined or uninterpretable percentage returns. In our raw dataset, the front-month WTI contract closed at $-\$37.63$ on 20 April 2020. We mask non-positive values in price columns as `NaN` and apply time-weighted interpolation to restore continuous, positive price series.

All executed operations, step-by-step missing counts, and row counts were logged directly to `report/evidence/cleaning_log.txt`.

5 Feature Strategy and Exploratory Data Analysis

5.1 Feature Engineering Strategy

Raw commodity prices and macroeconomic signals cannot be supplied directly to machine learning architectures without transformation. We construct temporal, autoregressive, rolling statistical, and cyclical features in `data_scripts/feature_engineering.py` using `pandas` [7]. All feature transformations use strictly contemporaneous or historical information to ensure no lookahead bias. Table 4 summarizes the engineered features.

Feature Name	Derivation	Predictive Rationale
`ret_<asset>`	Percentage change of asset price	Scale-free daily change across metals and signals
`ret_us10y`	First difference of 10-year Treasury yield	Change in percentage points (yield quoted in %)
`logret_gold`	Logarithmic difference $\ln(P_t) - \ln(P_{t-1})$	Continuous compounded returns for modeling
`gold_lag1/5/20`	Shift gold price by 1, 5, and 20 trading days	Autoregressive persistence over 1 day, 1 week, 1 month
`gold_ma7/ma30`	7-day and 30-day rolling arithmetic mean	Short-term trend momentum and medium-term baseline
`gold_vol30`	30-day rolling standard deviation of `ret_gold`	Dynamic return volatility and risk clustering
`month_sin/cos`	$\sin(2\pi m/12)$ and $\cos(2\pi m/12)$	Continuous cyclical monthly calendar encoding
`dow_sin/cos`	$\sin(2\pi d/5)$ and $\cos(2\pi d/5)$	Continuous cyclical trading day-of-week encoding
`gold_5d_ahead`	Negative shift by 5 trading days	Regression target: expected price 1 trading week ahead
`direction_5d`	Binary flag: (`gold_5d_ahead` > `gold`)	Classification target: 1 if future price rises, else 0

Table 4: Engineered features and forecasting targets. Forecasting targets are strictly excluded from model inputs.

We emphasize that `gold_5d_ahead` and `direction_5d` are forward-looking prediction targets and are never included as model inputs. Constructing the rolling windows and forward targets reduces the dataset from 4,225 to 4,190 rows: 30 leading rows are lost to the 30-day rolling windows and lags, and 5 trailing rows are dropped because they have no 5-day-ahead future price. The resulting dataset contains 29 columns, including the binary `is_outlier` flag.

5.2 Calendar Encoding and Categoricals

We encode calendar variables using cyclical sine and cosine transformations rather than one-hot encoding. One-hot encoding would add $12+5 = 17$ sparse binary columns and treat calendar intervals as mutually independent categories, discarding proximity. In contrast, trigonometric functions map calendar steps onto a continuous two-dimensional circle, preserving proximity between December and January as well as between Friday and Monday across the weekend. Because our dataset is structured in wide tabular time-series format, there are no text categories to one-hot encode, and `is_outlier` is already a native binary indicator.

5.3 Chronological Dataset Partitioning

To evaluate model generalisation in a time-series setting, we partition the dataset strictly chronologically without shuffling. Random partitioning or cross-validation would mix future observations into training folds, causing temporal data leakage. Table 5 details the three non-overlapping chronological partitions.

Split	Date Range	Rows	Share of `direction_5d == 1`
Train	2010-02-17 to 2021-12-31	2,988	54.25%
Validation	2022-01-03 to 2023-12-29	502	51.79%
Test	2024-01-02 to 2026-10-13	700	60.86%

Table 5: Chronological train, validation, and test partitions.

The training split provides nearly twelve years of historical data covering varied market regimes. The validation split serves for hyperparameter tuning and model selection, while the test split provides an out-of-sample holdout spanning 2024 to late 2026.

5.4 Outlier Capping and Feature Scaling

We apply return capping and feature scaling strictly on the training set to prevent data leakage. Data leakage occurs when statistics from validation or test periods inform preprocessing parameters, creating artificially optimistic evaluation metrics.

Return capping. We identify all daily return columns (`ret_*` and `logret_*`) and compute $3 \times$ IQR bounds ($[Q_1 - 3 \times \text{IQR}, Q_3 + 3 \times \text{IQR}]$) strictly on the training split. We then clip returns in all three partitions using these training bounds. Across the partitions, 176 values were clipped in Train, 22 in Validation, and 102 in Test. Capped unscaled features are preserved in `data/processed/features_unscaled.csv` for exploratory data analysis.

Feature scaling. Asset price levels trade near \$2,000 to \$4,000, whereas daily returns fluctuate around 0.01. Unscaled features would cause gradient-based estimators and distance metrics to be dominated entirely by nominal price magnitudes. We fit `StandardScaler` from `scikit-learn` [8] on the training partition across 22 continuous numerical features, then apply this fitted transformation to the validation and test sets. Targets (`gold_5d_ahead`, `direction_5d`), the binary outlier flag (`is_outlier`), and cyclical trigonometric encodings (`*_sin`, `*_cos`) are explicitly excluded from scaling. Min-max scaling will also be explored for recurrent neural network architectures (LSTM) in Deliverable 2.

Distribution shift limitation. Because gold prices rose substantially during 2024–2026 compared with the 2010–2021 training baseline, the post-scaling mean of gold in the test set is 8.35, while the validation mean is 1.94. These non-zero out-of-sample means are consistent with a train-only scaler operating under a structural regime shift. In Deliverable 2, we will evaluate whether forecasting returns or differenced series mitigates this nominal price scale divergence.

5.5 Exploratory Data Analysis

We performed exploratory analysis on the engineered and unscaled dataset to inspect relationships, class balances, distributional fat tails, seasonality, and volatility regimes.

Return correlations. Figure 6 displays the correlation matrix of daily percentage returns across metals and external indicators. Gold and silver exhibit the strongest positive co-movement with a daily return correlation of 0.79. In contrast, the US Dollar Index (`dxy`) displays an inverse correlation of −0.37 with gold, consistent with gold tending to move opposite to the dollar. The 10-year Treasury yield (`us10y`, measured as a daily change in percentage points) exhibits a negative correlation of −0.24 with gold, indicating that gold returns tend to be lower on days when yields rise.

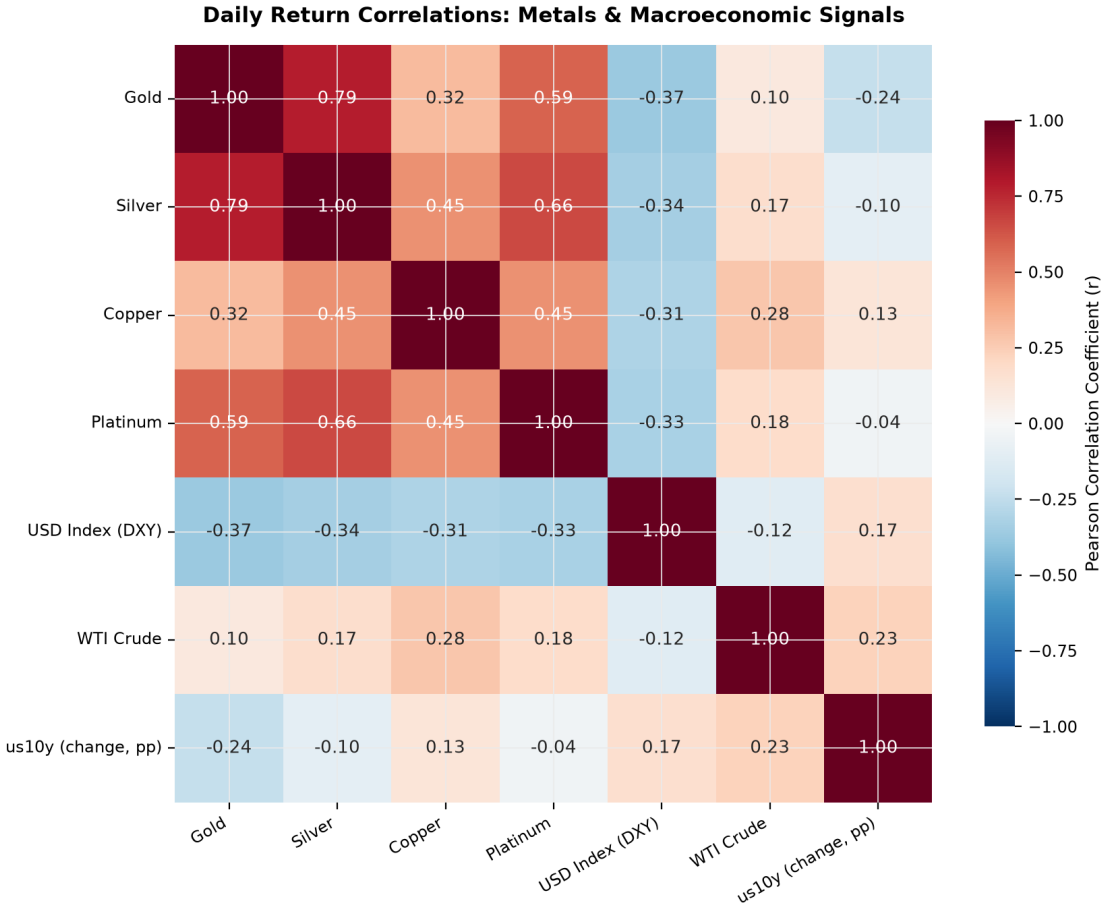


Figure 6: Correlation matrix of daily percentage returns and 10-year yield changes across metals and macroeconomic indicators.

Target class balance. Figure 7 plots target label frequencies across the chronological partitions. In the training split, 1,621 days were classified as Up (54.3%) and 1,367 as Down or Flat. In the validation split, 260 days were Up (51.8%) and 242 Down. In the out-of-sample test split, 426 days were Up (60.9%) and 274 Down. The target remains roughly balanced across train and validation, with a moderate upward shift in the test period reflecting recent market appreciation.

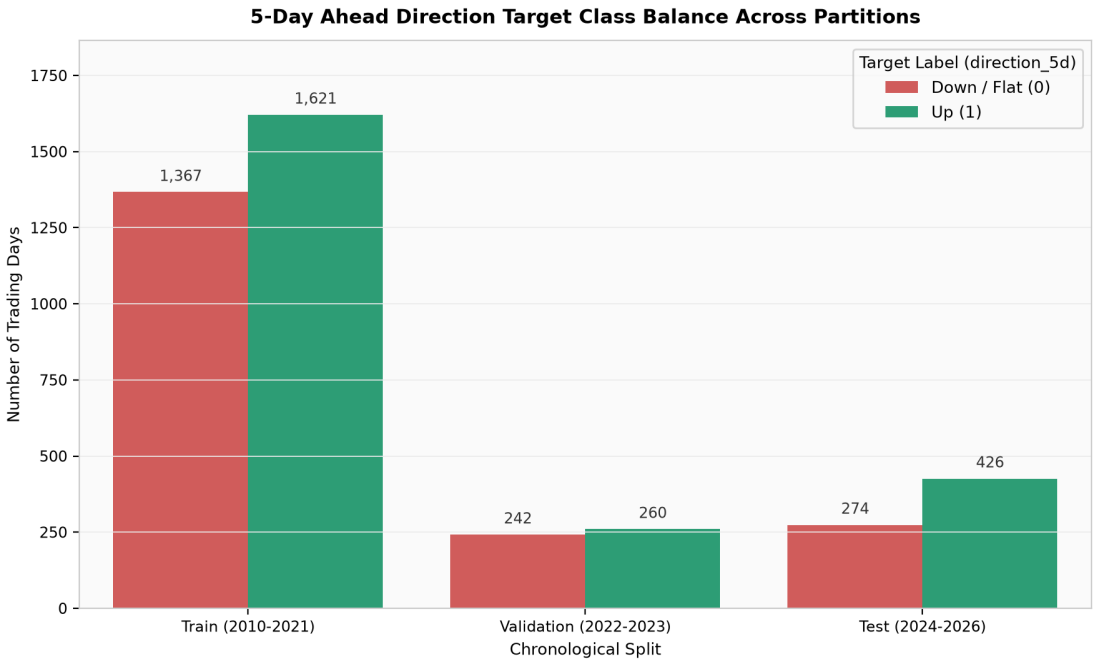


Figure 7: Class balance of the 5-day ahead directional target (`direction_5d`) across chronological partitions.

Return distributions and heavy tails. Figure 8 presents daily return histograms with kernel density estimates for gold, silver, and copper. In uncapped daily returns, all three metals exhibit severe excess kurtosis: +6.7 for gold, +15.6 for silver, and +13.8 for copper, accompanied by negative skewness (−0.60 for gold, −1.10 for silver, and −0.66 for copper). Applying the $3 \times$ IQR training-fitted cap reduces excess kurtosis to +1.7 for gold, +1.6 for silver, and +1.3 for copper, moderating extreme tail values while preserving the underlying distribution shape.

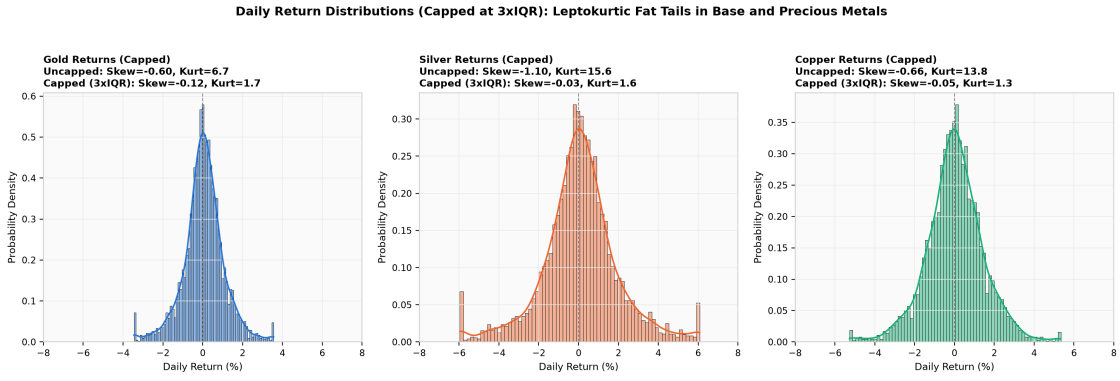


Figure 8: Daily return distributions for gold, silver, and copper under $3 \times$ IQR training-fitted capping.

Outlier profiles before and after capping. Figure 9 compares boxplots of raw returns against capped returns. Evaluating whisker outliers using the standard $1.5 \times$IQR rule yields 224 raw versus 226 capped outliers for gold (with 44 values resting at the cap), 281 raw versus 285 capped outliers

for silver (with 79 at the cap), and 200 raw versus 198 capped outliers for WTI crude (with 59 at the cap). Because capping enforces a $3\times$IQR threshold while boxplot whiskers extend to $1.5\times$IQR, clipped observations still lie beyond the boxplot whiskers, but their extreme magnitude has been bounded.

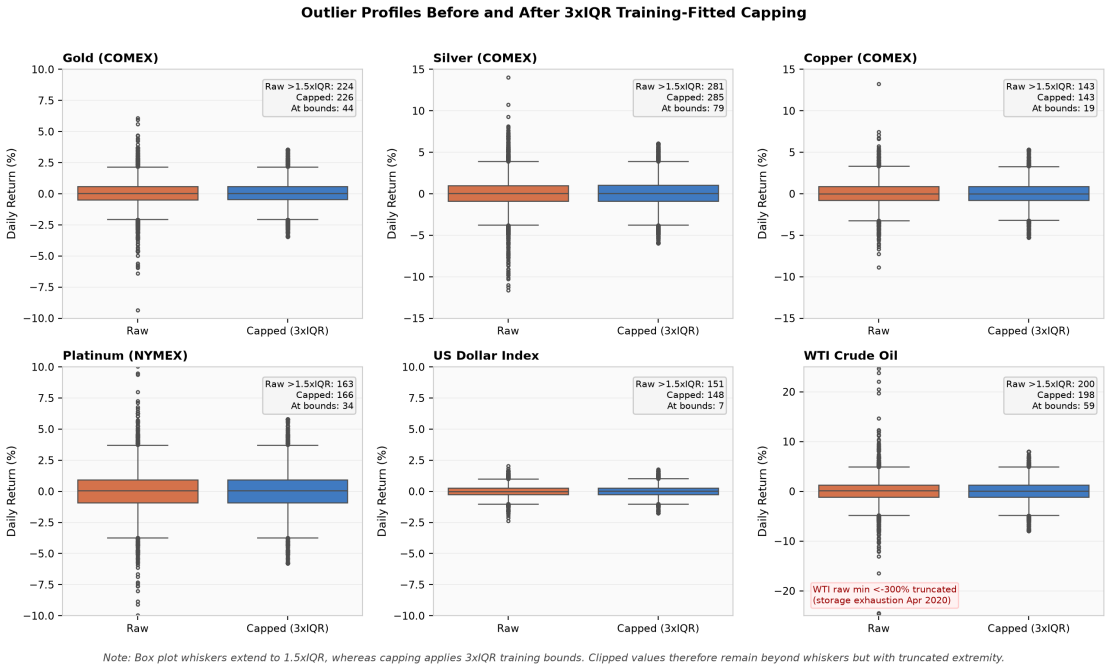


Figure 9: Return distributions across assets before and after $3 \times$ IQR training-fitted capping.

Seasonal decomposition. Figure 10 illustrates a multiplicative seasonal decomposition of monthly gold prices ($Y_t = T_t \times S_t \times I_t$, period 12) performed using `statsmodels` [9]. The estimated seasonal factor varies within a narrow band of 0.9797 to 1.0164, representing a maximum seasonal swing of about 3.67%. Over the sample period, the smoothed decomposition trend component rose by 262.1% (noting that the two-sided moving-average trend is undefined for the first and last six months), while raw gold closing prices rose by 269.8%. This indicates that, in our sample, the multi-year trend accounts for much more of gold's price movement than annual calendar seasonality.

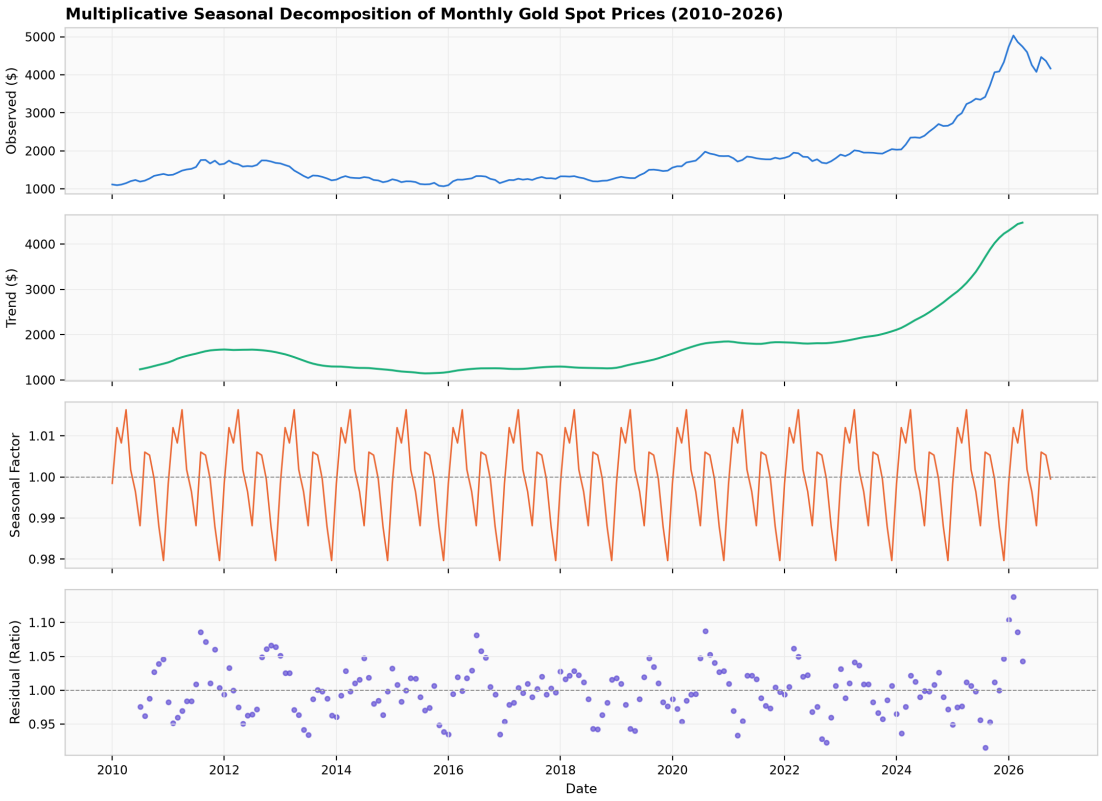


Figure 10: Multiplicative seasonal decomposition of monthly gold prices into trend, seasonal factor, and residuals.

Rolling volatility dynamics. Figure 11 plots the 30-day rolling standard deviation of daily gold returns over the 2010–2026 timeline. Over the sample, the median rolling daily volatility is 0.91%. The series reached a peak in our sample of 3.00% on 3 March 2026, alongside a COVID-period peak of 2.40% recorded on 24 March 2020. These peaks suggest that volatility varies across periods.

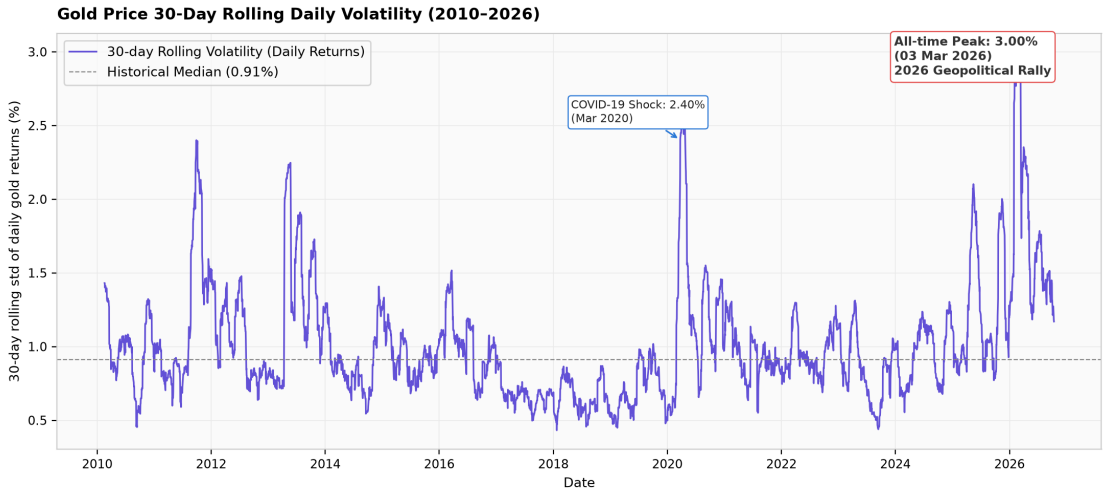


Figure 11: 30-day rolling standard deviation of daily gold returns (2010–2026) with annotated volatility peaks.

Gold sensitivity to the US Dollar. Figure 12 displays a scatter plot and ordinary least squares linear regression of daily gold returns against daily returns of the US Dollar Index. The estimated Pearson correlation is -0.369, with a regression slope of -0.857 and an R^2 of 0.136. While the negative slope indicates that gold returns tend to move opposite to the dollar, the modest R^2 indicates that currency movements explain only a small share of daily gold return variance, suggesting that other features are likely needed to explain gold returns.

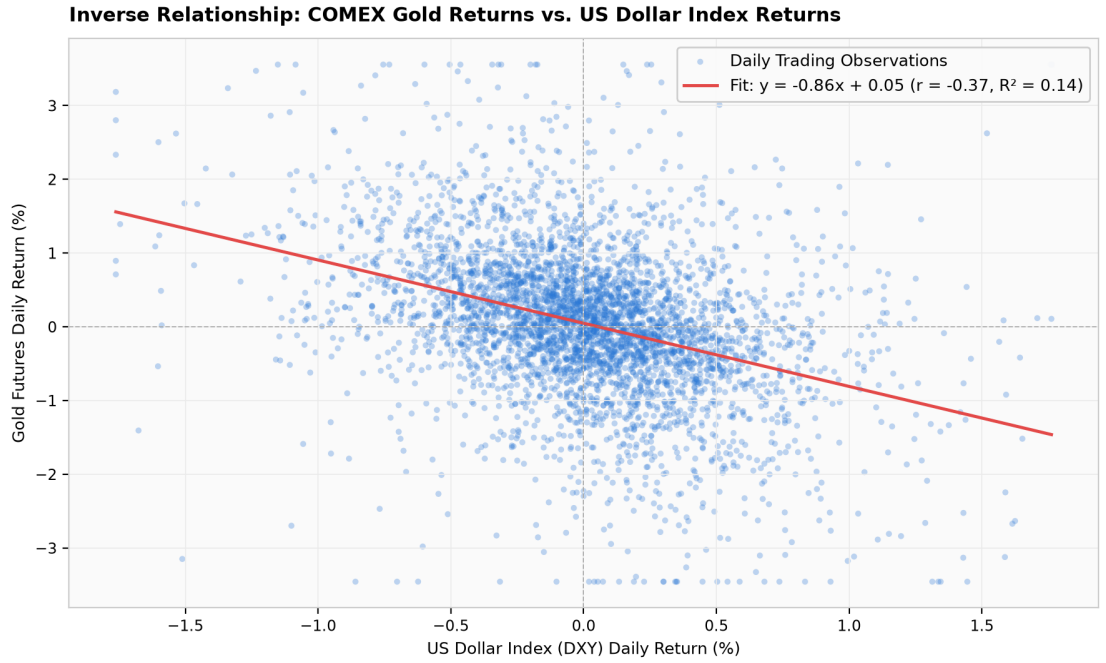


Figure 12: Scatter plot and linear regression fit of daily gold returns versus US Dollar Index returns.

6 Conclusion

This deliverable established the foundations of ProcurePulse. We defined the problem: small and medium-sized businesses that buy metals face large and unpredictable price movements, and cannot easily judge whether a supplier's quote is fair. We identified procurement managers at such businesses as the target users, and argued that machine learning is needed because metal prices change regime, depend on several interacting signals and require measured uncertainty, which fixed rules cannot provide.

We then built a reproducible data pipeline. Python scripts collect 16 years of daily data for four metals and four external signals from Yahoo Finance and the US Bureau of Labor Statistics, log every step, and fall back to backup sources when a source is unavailable. The raw data was merged into a single table of 4,300 rows, its quality problems were documented, and it was then cleaned, turned into model-ready features and explored visually (Sections 4 and 5).

In the next deliverable we will train and compare the forecasting models described in Section 2: a naive baseline, ARIMA, Prophet, XGBoost and an LSTM. We will evaluate their accuracy and prediction intervals on the held-out test period, measure whether the external signals improve the forecasts, and use the best model to produce the first Buy/Hold/Negotiate recommendations.

References

[1] Ran Aroussi. yfinance: Download market data from yahoo! finance's api. `https://github.com/ranaroussi/yfinance`. Python library, accessed October 2026.

[2] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. *Time Series Analysis: Forecasting and Control*. Wiley, 5th edition, 2015.

[3] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[4] Federal Reserve Bank of St. Louis. FRED, federal reserve economic data. `https://fred.stlouisfed.org/`. Accessed October 2026.

[5] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[6] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021.

[7] Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.

[8] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[9] Skipper Seabold and Josef Perktold. statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010.

[10] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018.

[11] U.S. Bureau of Labor Statistics. Public data API, version 2.0. `https://www.bls.gov/developers/`. Series CUSR0000SA0, accessed October 2026.